

Security Audit

MyShell (dApp)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	7
Intended Smart Contract Behaviours	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	21
Conclusion	22
Our Methodology	23
Disclaimers	25
About Hashlock	26

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Myshell partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed contracts were secure.

Project Context

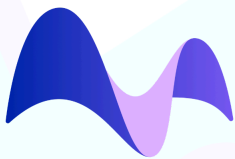
MyShell is a decentralized AI platform connecting consumers, creators, and researchers. Its new feature, the Patron Badge, encourages interactions between patrons and AI creators by allowing patrons to earn shared Points by spending Shell Coins on AI app badges. This system boosts creators' exposure and reputation as their apps gain value.

Project Name: MyShell team

Compiler Version: ^0.8.0

Website: <https://myshell.ai/>

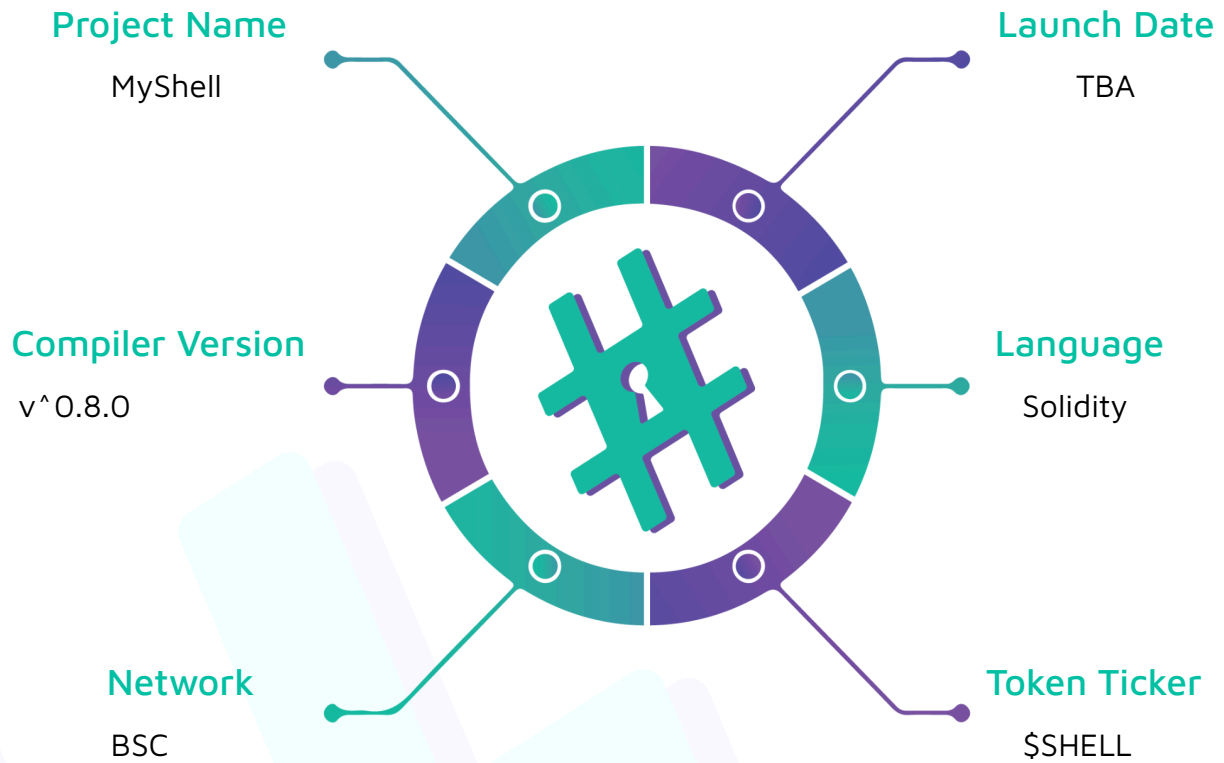
Logo:



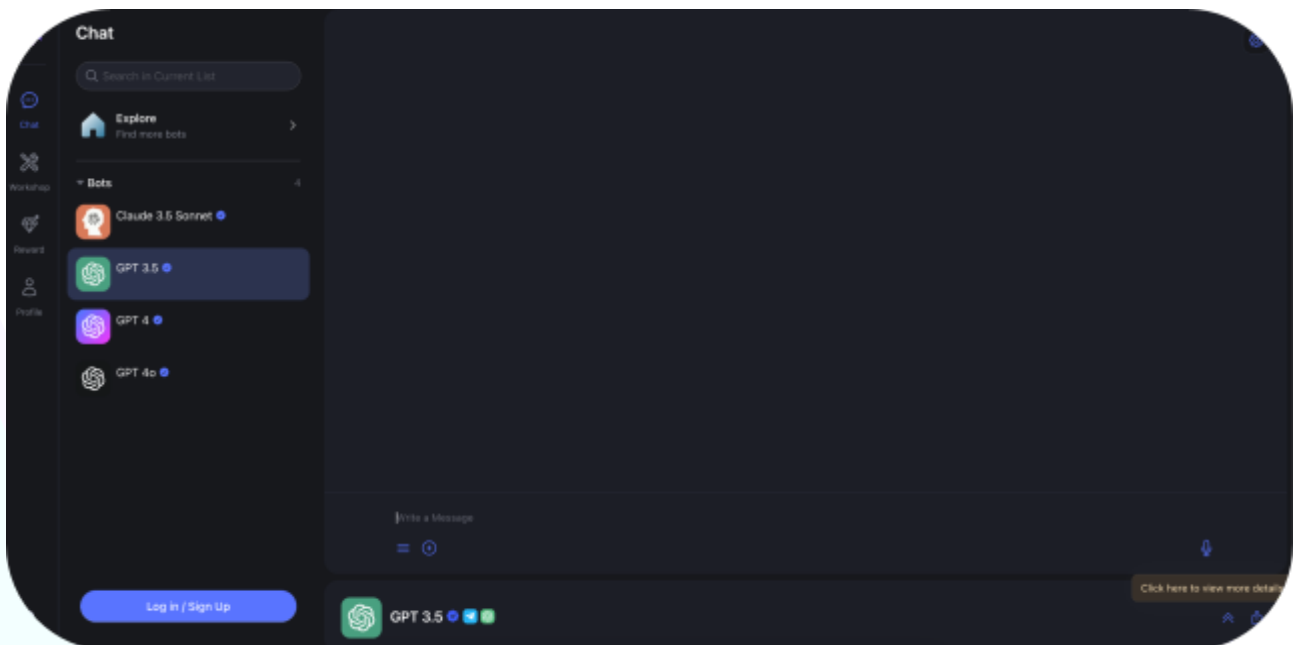
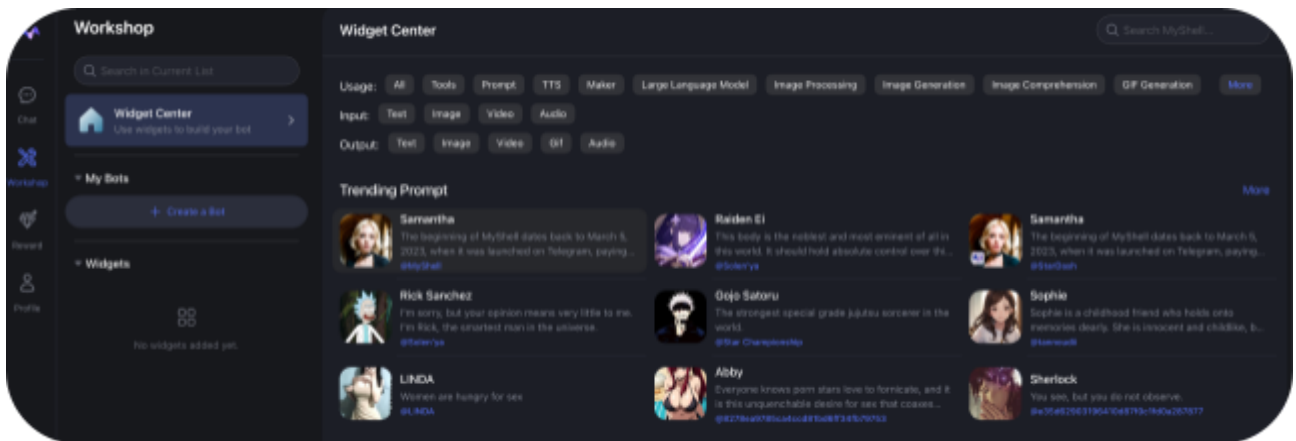
MyShell

#Hashlock.

Hashlock Pty Ltd

Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the MyShell project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	MyShell Protocol Smart Contracts
Platform	BSC / Solidity
Audit Date	July, 2024
Contract 1	BondingCurve.sol
Contract 1 MD5 Hash	2f47e39870bb0fa83cdcc312dc6b3fd5
Contract 2	ViewHelper.sol
Contract 2 MD5 Hash	659f681ba4c7274d3d5079bd920920eb

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We have identified some significant vulnerabilities that need to be addressed prior to launch.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section.

We initially identified some significant vulnerabilities that have since been addressed.

Hashlock found:

0 High-severity vulnerabilities

1 Medium-severity vulnerabilities

8 Low-severity vulnerabilities

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Behaviours

Claimed Behaviour	Actual Behaviour
BondingCurve.sol - Allows users to: - Launch new bondings - Buy shares in bondings - Sell shares in bondings - Allows admins to: - Set protocol fee destination - Set various fee percentages - Set governance address - Set operators - Blacklist bondings - Set hold limit, open permit after, and permitted share - Upgrade the contract	Contract achieves this functionality.
ViewHelper.sol - Allows users to: - Get user information regarding bondings	Contract achieves this functionality.

Code Quality

This Audit scope involves the smart contracts of the MyShell Badge project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the MyShell project smart contract code as a zip file.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow and the complex code logic. The comments help us understand the overall architecture of the protocol.

Dependencies

Per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry-standard open-source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

Medium

[M-01] BondingCurve#blacklistBid - Accidentally blacklisted bondings cannot be whitelisted leading to loss of fee income for the bonding initiators

Description

The `blacklistBid` function allows bondings to be blacklisted. However, if a bonding is blacklisted accidentally, there's no way to whitelist the bonding back leading the loss of earnings from the fees for the bonding initiators.

Vulnerability Details

The `blacklistBid` function adds the given bonding ID (`bid`) to `_bondingBlacklist` mapping:

```
function blacklistBid(uint bid, bool blacklisted) external onlyGov {  
    _bondingBlacklist[bid] = true;  
    emit LogBlacklistBonding(bid, _bondingInitiator[bid], blacklisted);  
}
```

The current implementation only allows adding a bonding ID to the blacklist but does not provide any mechanism to remove or whitelist it back. If a bonding is blacklisted accidentally, it cannot be undone, causing permanent restrictions.

Impact

Accidentally blacklisted bondings cannot be whitelisted, leading to permanent loss of fee income for bonding initiators .

Recommendation

To address these issues, the `blacklistBid` function should be modified to allow for whitelisting of bondings, providing a way to reverse the blacklist status if needed. This

can be achieved by setting the `_bondingBlacklist[bid]` to the value of the `blacklisted` parameter:

```
function blacklistBid(uint bid, bool blacklisted) external onlyGov {  
    _bondingBlacklist[bid] = blacklisted;  
    emit LogBlacklistBonding(bid, _bondingInitiator[bid], blacklisted);  
}
```

Status

Resolved.

Low

[L-01] Contracts - Code does not follow the best practice of check-effects-interactions pattern

Description

Code should follow the best-practice of CEI, where state variables are updated before any external calls are made. Doing so prevents a large class of reentrancy bugs.

There is 1 instance of this issue:

./contracts/BondingCurve.sol:buyBonding

```
if(msg.value > priceAfterFee) payable(msg.sender).transfer(msg.value - priceAfterFee);
```

Recommendation

To enhance the security of your smart contract and prevent reentrancy attacks, it's crucial to adhere to the check-effects-interactions pattern. Restructure your contract's functions to follow this pattern by first performing condition checks (checks), then updating the contract's state (effects), and finally, executing any external calls or interactions (interactions). This sequential approach reduces the risk of reentrancy vulnerabilities by ensuring that state changes are isolated from external interactions and condition checks are made before any state modifications. Review and refactor your contract code to align with this best practice for improved security.

Status

Acknowledged.

Directly transferring ether won't be a problem due to onlyEOA modifier. Adding nonReentrant modifier only consumes redundant gas.

[L-02] Contracts - Missing checks for address(0) in constructor/initializers/functions

Description

In Solidity, the Ethereum address `0x0000000000000000000000000000000000000000` is known as the "zero address". This address has significance because it's the default value for uninitialized address variables and is often used to represent an invalid or non-existent address.

The "Missing zero address control" issue arises when a Solidity smart contract does not properly check or prevent interactions with the zero address, leading to unintended behavior. For instance, a contract might allow tokens to be sent to the zero address without any checks, which essentially burns those tokens as they become irretrievable. While sometimes this is intentional, without proper control or checks, accidental transfers could occur.

There is 8 instance of this issue:

./contracts/ViewHelper.sol:constructor

```
constructor(address bc) {  
    _bc = IBondingCurve(bc);  
}
```

./contracts/BondingCurve.sol:BondingCurve_init

```
function BondingCurve_init(  
    address protocolFeeDestination_  
) public virtual initializer {  
    ... REDACTED FOR BREVITY ...  
    _protocolFeeDestination = protocolFeeDestination_;  
}
```

./contracts/BondingCurve.sol:setGov

```
function setGov(address gov) external onlyOwner {  
    _gov = gov;  
}
```

Recommendation

It is strongly recommended to implement checks to prevent the zero address from being set during the initialization of contracts. This can be achieved by adding require statements that ensure address parameters are not the zero address.

Status

Acknowledged.

- ViewHelper.sol is a reading helper contract which can be replaced anytime, so no check for _bc parameters in constructor will be fine.
- As for BondingCurve.sol.setGov(), the gov will be set as zero address when the owner doesn't need gov to manage parameters. This feature is designed intentionally.

[L-03] Contracts - Do not allow fees to be set to 100%**Description**

It is recommended from a risk perspective to disallow setting 100% fees at all. A reasonable fee maximum should be checked for instead.

There is 4 instance of this issue:

- **./contracts/BondingCurve:setProtocolFee**
- **./contracts/BondingCurve:setInitiatorFee**

Recommendation

Implement strict validation checks in your smart contract's fee-setting functions to ensure that fees cannot be configured to 100%. Define a maximum allowable fee percentage that preserves the contract's functionality and user experience, such as 10% or 20%, depending on your project's specific requirements and economic model. For example:

```
function setTransactionFee(uint256 newFeePercentage) public onlyOwner {
    require(newFeePercentage <= 20, "Fee cannot exceed 20%");
    transactionFee = newFeePercentage;}

```

Status

Acknowledged.

- There is no upper limit by design, so the gov will have the max privilege to set the value, yet the value will be reasonable.

[L-04] Contracts - Use of "transfer" instead of "call" to send native assets

Description

The use of `transfer()` in the contracts may have unintended outcomes on the native asset being sent to the receiver. The transaction will fail when:

- The receiver address is a smart contract that does not implement a payable function.
- The receiver address is a smart contract that implements a payable fallback function which uses more than 2300 gas units.
- The receiver address is a smart contract that implements a payable fallback function that needs less than 2300 gas units but is called through proxy, raising the call's gas usage above 2300.
- Additionally, using a gas value higher than 2300 might be mandatory for some multi-signature wallets.

There are 10 instances of this issue:

./contracts/BondingCurve.sol:

```

178:if (msg.value > priceAfterFee) payable(msg.sender).transfer(msg.value -
priceAfterFee);

211:payable(msg.sender).transfer(priceAfterFee);

221:if(!_bondingBacklist[bid])payable(_bondingInitiator[bid]).transfer(initiatorFee);
else protocolFee += initiatorFee;

226:payable(_protocolFeeDestination).transfer(protocolFee);

```

Recommendation

To transfer, a native asset is recommended to either use:

- <receiver>.call.value(amount)
- OpenZeppelin Address.sendValue()

Status

Acknowledged.

onlyEOA makes sure smart contract won't be able to interact with BondingCurve.sol. And currently, interactions from either Account Abstraction or smart contracts are not necessary.

[L-05] Contracts - Vulnerable versions of packages are being used

Description

This project is using specific package versions which are vulnerable to the specific CVEs listed below. Consider switching to more recent versions of these packages that don't have these vulnerabilities.

List of the vulnerabilities for the v4.7.3 OpenZeppelin contracts library:

- CVE-2023-40014 - MEDIUM - (openzeppelin/contracts >=4.0.0 <4.9.3)
- CVE-2023-34459 - - (openzeppelin/contracts >=4.3.0 <4.8.3)
- CVE-2023-34234 - MEDIUM - (openzeppelin/contracts >=4.3.0 <4.8.3)
- CVE-2023-30542 - HIGH - (openzeppelin/contracts >=4.3.0 <4.8.3)
- CVE-2023-30541 - MEDIUM - (openzeppelin/contracts >=3.2.0 <4.8.3)

Recommendation

Consider updating packages to safest version.



Status

Resolved.

[L-06] Contracts - tx.origin == msg.sender control blocks MultiSig wallets

Description

The use of the control check `tx.origin == msg.sender` in certain functions of the smart contract can impede multisig wallet users. This security measure, while preventing certain types of attacks, also restricts multisig wallets from calling these functions, as the immediate sender (the multisig contract) differs from the original transaction initiator.

There is 1 instance of this issue:

* **./contracts/BondingCurve.sol**

```
52: require(tx.origin == msg.sender, "not EOA");
```

Recommendation

Consider revising the contract logic to accommodate multisig wallet interactions, perhaps by implementing alternative security measures or allowing specific multisig wallets. This adjustment would maintain security while enhancing usability for multisig wallet users.

Status

Acknowledged.

tx.origin == msg.sender, this strict condition helps make sure users are actually EOA accounts.

[L-07] Contracts - Non Disabled Implementation Contract

Description

The upgradeable contracts do not disable initializers in the constructor, as recommended by the imported Initializable contract. This means that anyone can call the initializer on the implementation contract to set the contract variables and assign the roles. To reduce the potential attack surface, `_disableInitializers` in the constructor needs to be called.

There is 1 instances of this issue:

* `./contracts/BondingCurve.sol`

Recommendation

Build a constructor function in the upgradeable contracts that calls the `disableInitializers()` function.

Status

Resolved.

[L-08] Contracts - Incorrect Use of EIP-712 Domain Separator

Description

The `DOMAIN_SEPARATOR` is only derived once in the initializer, which may lead to issues if the contract is upgraded and the domain separator should be updated. It should be re-derived on-demand or re-initialized properly.

There is 1 instance of this issue:

* `./contracts/BondingCurve.sol`

Recommendation

It is recommended to re-derive the `DOMAIN_SEPARATOR` dynamically or during the upgrade process to ensure it remains accurate.

Status

Acknowledged.

When upgraded, version will be changed and DOMAIN_SEPARATOR will be updated based on the new version.

Centralisation

The MyShell values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the MyShell Badge project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au

#Hashlock.



#Hashlock.

Hashlock Pty Ltd